Man-in-the-Middle Attack on HTTPS: TLS Proxying with Forged Certificates

Oleksandra Popovych
s0231589@rz.uni-frankfurt.de
Goethe University
Frankfurt (Main), Germany

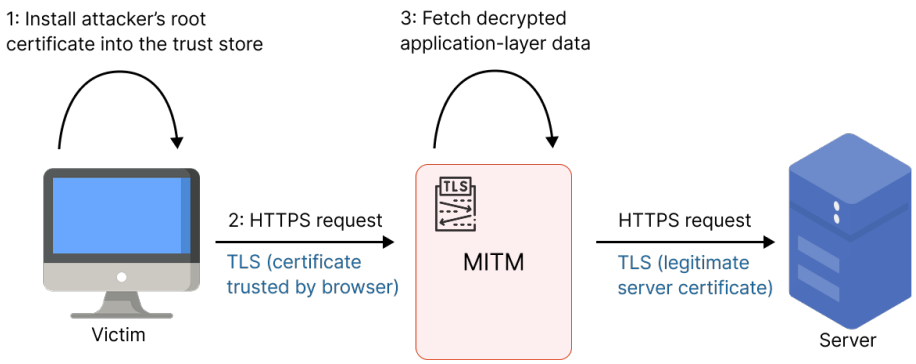

Figure 1: MITM attack scenario for HTTPS traffic interception.

Abstract

Despite growing cybersecurity measures and increasing user awareness, attackers continue to succeed in compromising large amounts of sensitive data online. This paper examines in detail a practical and still highly relevant threat: the Man-in-the-Middle (MITM) attack, in which an attacker positions themselves between a client and a server to intercept and manipulate communication. Several techniques fall under the MITM category. This work focuses on a MITM attack against SSL/TLS that exploits weaknesses in certificate trust assumptions. Throughout the attack, the victim remains unaware, as the attacker does not disrupt the connection but silently intercepts and observes the exchanged data. The paper also discusses practical security measures that can mitigate the risks associated with this attack.

Keywords

Man-in-the-Middle Attack, SSL/TLS Security, Certificate Authority Trust, Session Hijacking, Network Security

1 Introduction

With the rapid expansion of the internet, nearly every online user has become a potential target for cyberattacks. Despite increasing awareness and advances in cybersecurity, malicious actors continuously adapt and refine their methods. In 2024, in the United States alone, 3.158 data-compromise incidents were recorded, affecting more than 1.35 billion people through data breaches, leaks, and exposures.[8] Today, hackers can exploit even the smallest vulnerabilities or misconfigurations in software systems. Data-transfer networks are especially attractive targets, since enormous amounts of sensitive and private information are transmitted across them.

One of the most common network-based attacks is the Man-in-the-Middle (MITM) attack, in which an attacker positions themselves between a client and a server in order to intercept communication. More precisely, a MITM attack exploits weaknesses in trust assumptions at different layers, including user trust in local networks, certificate trust decisions, and reliance on browser security indicators to assess the authenticity of a website. As a result, after a login attempt in such a scenario, the attacker may capture submitted sensitive information and observe a large part of the incoming traffic. Interestingly, throughout the history of MITM attacks, many security breaches have occurred due to human element. In 2023, when Verizon completed the Data Breach Investigations Report for Yahoo, the report showed that 74% of all breaches involved a human aspect, such as privilege misuse, social engineering, or stolen credentials.[1]

HTTPS uses a secure channel to transfer data between a client and a server. This channel is established using the TLS protocol, which replaced the older SSL protocol. For this reason, the suffix "S" in HTTPS denotes the use of TLS-based encryption. Certificate validation ensures that a connection is established with a legitimate and trusted HTTPS website. When this validation fails, the browser displays a certificate error and may allow the user to decide whether to proceed, typically at their own risk. Although HTTPS is generally considered secure, its certificate infrastructure introduces potential points of compromise. Attackers may exploit weaknesses in this trust model or compromise real-world certificate authorities (CAs) to conduct MITM attacks.[3]

Figure 1 illustrates the step-by-step process of a possible MITM attack scenario:

(1) The victim receives and installs the attacker's root certificate into the browser trust store.

(2) When the victim accesses an HTTPS-protected web service, the connection is routed through an attacker-controlled intermediary, which acts as a TLS endpoint using a certificate trusted by the browser.
(3) As the victim authenticates, the attacker, positioned as a MITM, can observe decrypted application-layer data, including authentication credentials.

Several attack methods fall under the MITM category. This work focuses on a MITM attack against SSL/TLS that exploits vulnerabilities in the certificate trust model. While the MITM approach itself is not novel, many existing experimental studies rely on tools and techniques with limited effectiveness in modern browser environments and focus primarily on HTTP traffic. Therefore, this paper examines more recent MITM approach, analyzes how contemporary websites respond to attacks on HTTPS, and discusses potential countermeasures to mitigate their impact.

The next section reviews related work, followed by an overview of TLS certificate validation and HTTPS communication, including how encrypted traffic can be intercepted under compromised trust assumptions. The paper then examines the MITM threat model, describes the experimental setup, discusses countermeasures, and concludes with final observations.

2 Related Work

Clark et al. (2013) [3] summarized and categorized several prominent weaknesses in the HTTPS ecosystem and CA infrastructure, including weak encryption configurations, the large number of trusted CAs capable of issuing forged certificates, inadequate certificate validation procedures, and insufficient reliability of browser security warnings. They conclude that while it is extremely difficult for attackers to break the TLS protocol and its cryptographic mechanisms directly, attackers may instead shift their focus to more vulnerable aspects of HTTPS, particularly the CA infrastructure combined with human factors. While Clark et al. discuss issues related to CA primarily from a conceptual perspective, de Carné de Carnavalet and Mannan (2016)[5] present a more concerning practical use case demonstrating how certificate forging is employed by certain antivirus and parental-control applications. These tools install their own root certificate into the client's trust store and dynamically generate forged certificates to impersonate remote servers, thereby enabling interception of HTTPS traffic by the MITM.

Chordiya et al. (2018)[2] provide an in-depth analysis of the MITM attack model over secure network connections, demonstrating how ARP Spoofing can be used to obtain a MITM position and how SSL Stripping can then be applied to downgrade encrypted traffic. Their study includes a successful real-world experiment in which victim traffic is transparently intercepted and rerouted through the attacker's machine. However, the SSL stripping approach applied by Chordiya et al. has limited effectiveness for downgrading encrypted traffic in modern browsers due to widespread adoption of HTTPS and mechanisms such as HSTS. Moreover, many users report stability issues and implementation problems when using tools such as sslstrip. In contrast, this paper focuses on MITM attacks targeting SSL/TLS directly, with particular attention to how modern websites respond to such attacks and which indicators may alert the victim to abnormal behavior.

3 Background

Several aspects of network and web architecture contribute to the feasibility of MITM attacks. This section describes the underlying factors that attackers can exploit to perform such attacks.

3.1 TLS Handshake

TLS consists of two main protocol components: the handshake protocol and the record layer. During the handshake phase, the client and server negotiate the cryptographic parameters of the connection, including the encryption algorithms to be used. The handshake also performs authentication between the client and the server and establishes a shared session key. The two most commonly used key exchange mechanisms in TLS are RSA-based key transport and authenticated Diffie–Hellman (DH) key exchange. In both RSA and authenticated DH cipher suites, the server provides its public key to the client within the Certificate message. In RSA-based key exchange, the client encrypts a pre-master secret using the server's public key, and only the server holding the corresponding private key can decrypt it. In contrast, in authenticated DH key establishment, both parties independently compute a shared secret through the DH exchange, while the server's certificate is used solely to authenticate the key exchange via a digital signature.[9]

To perform content inspection on employee machines, corporate IT environments may install their own browser-trusted root certificates within a private network, as an internal trust anchor. This practice, while outside the public CA trust model, enables HTTPS connections to be proxied and inspected by specially designed middleware operating as a MITM. [3] Similarly, some antivirus and parental-control applications employ the same approach to filter user traffic (e.g., blocking access to unwanted websites for children). This is generally achieved by placing a proxy in a MITM position between the browser and the web server. To enable this, the software installs its own root certificate into the client's trust store, thereby allowing it to act as a trusted certificate authority. When the client initiates a connection to a server, the TLS proxy generates and presents a substitute certificate for the target domain. As a result, the client's TLS session terminates at the proxy, while a second TLS connection is established between the proxy and the remote server. This setup allows the proxy to decrypt, inspect, and forward all data exchanged between the client and the server. [5]

3.2 Certification

A web certificate establishes a binding between a domain name and a public key. This binding is asserted and maintained by a trusted CA through a digital signature. The corresponding private key is used locally by the server to authenticate itself during the TLS handshake and to establish a secure communication channel. In general, the CA model is designed to prevent MITM attacks by enabling clients to verify the authenticity of third parties through cryptographic signatures on server certificates. If certificate validation fails, the client terminates the connection. [3]

Some software tools (e.g.,*mitmproxy*) circumvent this protection by installing themselves as trusted CAs, allowing them to

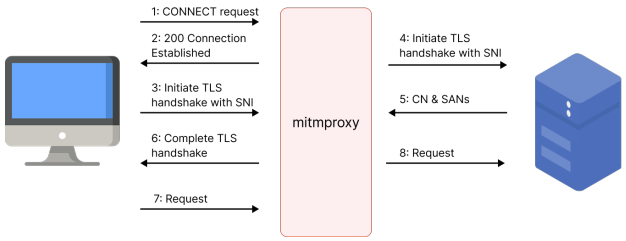

Figure 2: How *mitmproxy* works: Explicit HTTPS. [4]

generate certificates that are accepted by the client. To achieve this, the software requires the corresponding domain name. For instance,*mitmproxy* obtains it through upstream certificate sniffing by briefly establishing a TLS connection to the target server, extracting the Common Name (CN) from the server's certificate, and using it to generate a matching substitute certificate for the client. Additionally, a Subject Alternative Name (SAN) extension is included in the certificate, allowing alternative domain names to be specified and enabling the connection to proceed even when the domain does not match the certificate's CN. Multiple domains often share a single IP address, which is handled through the Server Name Indication (SNI) extension. SNI allows the client to indicate the intended hostname during the TLS handshake. SNI is essential because connecting to a server without providing SNI may result in receiving a default certificate unrelated to the client's intended destination.[4]

Figure 2 illustrates the underlying concept of TLS proxying for HTTPS using *mitmproxy* as an example:

(1) The client initiates a connection to *mitmproxy* by issuing an HTTP CONNECT request.
(2) *mitmproxy* responds with an HTTP 200 Connection Established status.
(3) The client initiates a TLS session and indicates the target hostname using the SNI extension.
(4) Based on the received SNI value, *mitmproxy* establishes a TLS connection to the target server.
(5) After receiving the server's certificate, *mitmproxy* extracts the CN and SAN values to generate a matching substitute certificate.
(6) The TLS handshake between the client and *mitmproxy* is then resumed and completed using the generated certificate.
(7) The client sends subsequent requests over the established TLS connection.
(8) *mitmproxy* forwards these requests to the target server over a separate TLS connection.[4]

4 Threat Model

Before the malicious intrusion, the attacker ensures that the target service uses HTTPS and relies on standard browser certificate validation, that the victim's browser accepts certificates signed by any trusted root in its trust store, and that the victim can be induced to install an attacker-controlled root certificate. During the attack, the attacker positions themselves as an on-path intermediary between the victim and the target web service via a proxy configuration.

Once the attacker-controlled certificate is trusted by the browser, the attacker can intercept and decrypt HTTPS traffic while maintaining normal connectivity to the target service. The victim does not distinguish between a legitimate server certificate and a substitute certificate signed by a trusted root. The attacker's goal is to convince the victim that the session is secure and thereby obtain sensitive information, such as authentication credentials or payment details.

5 Experiment

For research purposes, an experimental setup consisting of two virtual machines was used: Kali Linux 2025.3, acting as the attacker machine, and Ubuntu 25.10, representing the victim. The victim machine used Mozilla Firefox version 147.0.1 as the web browser. To establish a MITM position and intercept HTTPS traffic, the open-source TLS interception proxy *mitmproxy* was deployed on the Kali Linux system. In each phase of the experiment, we analyze the behavior of modern browsers during the attack and the mechanisms they use to warn users about potential security risks. These are discussed in detail later in the Results section. The exact steps and pseudo code for the attack experiment are presented below:

(1) Start mitmproxy (UI version) on the attacker machine: `mitmweb`
(2) In the capture section, set the explicit HTTPS proxy to listen on the attacker's IP address and port 8080.
(3) On the victim machine's browser, manually configure proxy access to the Internet by adding the attacker machine's IP address as the HTTP(S) proxy.
(4) To install the certificate, access `http://mitm.it`. The website must be accessible and display certificate download options.
(5) Download the Firefox certificate. In the Firefox browser, go to `Settings > View Certificates` and import the downloaded certificate.
(6) Trust this CA to identify websites.
(7) On the victim's machine, attempt to log in to a well-known e-commerce platform.
(8) Observe the unencrypted credentials on the attacker's machine in the `mitmweb` flow list.

6 Results

When visiting the e-commerce platform before the attack, it is accessible and uses HTTPS (see Figure 3). The website's certificate is issued by the widely trusted CA DigiCert Inc. The browser indicates that the connection is secure, the certificate is valid, and the communication is encrypted.

Next, after the attacker starts *mitmproxy* and the victim switches to the attacker machine's IP address as the HTTP(S) proxy, the victim attempts to connect to the website again (see Figure 4). However, this time the browser warns the user about possible issues with the connection, stating that a software application (*mitmproxy*) is preventing Firefox from safely connecting to the site. The warning also mentions that the website has a security policy called HTTP Strict Transport Security (HSTS), which means that Firefox can only connect to it securely and that no exception can be added

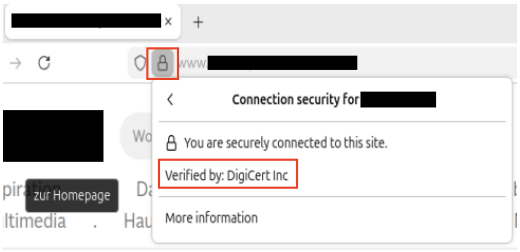

Figure 3: Connection security for a well-known e-commerce platform before the attack.

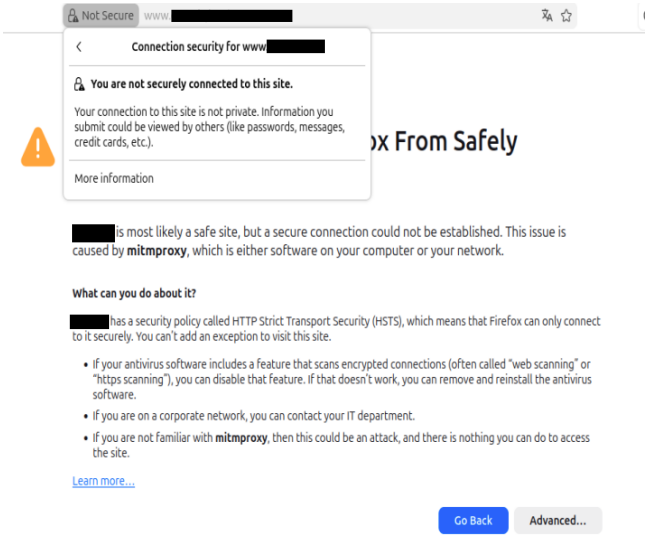

Figure 4: Security warning for an e-commerce platform during the proxying.

for this site. The icon to the left of the browser's address bar also indicates that the connection is not secure.

After the victim imports the certificate provided by the attacker into the browser's trust store and chooses to trust the CA to identify websites, the victim attempts to connect to the e-commerce platform again. This time, the connection is successfully established and the website is accessible, with no warnings displayed (see Figure 5). The user can navigate normally and submit requests as usual. The lock icon does not indicate any suspicious behavior. Only after clicking on the lock icon does the first hint appear: the connection is verified by a certificate issuer that is not recognized by Mozilla. Nevertheless, the connection is still reported as secure. When inspecting the connection details further, another statement is shown, indicating that the certificate is verified by *mitmproxy* and that the certificate may have been added by the operating system or an administrator.

At this point, the victim may attempt to use their login credentials on the website. When the data is submitted, the attacker can inspect the flow list on their machine and identify the corresponding POST request sent by the victim from the e-commerce platform. The attacker collects the data, while the victim continues to use the website without any noticeable issues (see Figure 6).

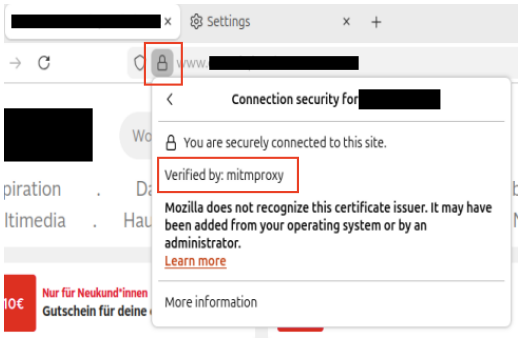

Figure 5: Connection security for an e-commerce platform after adding mitmproxy certificate to the trust store.

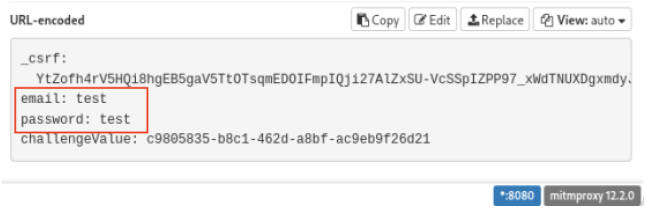

Figure 6: Collect victim's login credentials.

7 Countermeasures

The experiment conducted in this paper shows that modern browsers and websites still have limited ability to warn users about suspicious connection behavior and unexpected certificate substitution, which can allow attackers to access unencrypted sensitive traffic. Nevertheless, there exist some simple and practical protection mechanisms that can be applied by users in almost any environment.

One such mechanism is key pinning based on the client's browsing history. A pinning mechanism records the public key previously accepted by the browser for a particular website and raises a warning if the public key changes unexpectedly. Such functionality can be found, for example, in the Firefox extension Certificate Patrol.[3] Another countermeasure can be applied directly by domain owners, who can make use of publicly searchable logs of issued certificates. By whitelisting only valid TLS certificates and authorized certificate authorities for their domains, and rejecting all others, domain owners can detect and prevent the use of illegitimately issued certificates. This approach enables continuous monitoring and early detection of unauthorized certificates associated with a specific domain.[3]

During the experiment, it was also observed that some websites, such as `www.facebook.com` , unlike the website tested in this work, do not transmit user passwords in plaintext to the server but instead apply additional client-side processing before submission. As a result, even if a MITM attacker can obtain the username in plaintext, the password remains protected in a transformed form. Joshi et al. [7] describe a possible password protection technique against MITM attacks that proposes hashing (e.g., using SHA-1) the user's password together with SSL certificate parameters via a client-side

script before submitting it to the website over an HTTPS endpoint. Their approach also makes use of information embedded in the server's PKI certificate.

Another secure approach to protecting passwords on the client side is the use of Asymmetric Password-Authenticated Key Exchange (aPAKE) protocols. These protocols are designed to provide secure password authentication and key exchange between a client and a server without relying on the traditional PKI model and without disclosing the user's password to the server or any intermediate entity. OPAQUE is one such protocol, providing protection against password leakage while also ensuring that the password remains hidden from the server, even during the password registration phase.[6]

8 Conclusion

In this paper, a still highly relevant MITM attack on SSL/TLS based on forged certificates and TLS proxying was described and examined. At the same time, the behavior of contemporary websites under attacks on HTTPS was analyzed. Furthermore, the paper discussed potential countermeasures that can significantly mitigate such attacks and, even when an attacker is already positioned in the middle, make it extremely difficult to obtain sensitive data in plaintext.

The main results of the experiment show that if a victim is not sufficiently cautious and can be induced to install an attacker-controlled root certificate, an attacker can make the victim believe that the connection remains secure and can trick the victim into disclosing credentials by authenticating to a service. Meanwhile, neither the browser nor the web service explicitly warns the user that an unusual security event has occurred. This highlights the need for the discussed countermeasures to be treated not as optional enhancements, but as necessary protections. In particular, stronger certificate monitoring mechanisms, defensive browser extensions, and client-side password protection techniques should be considered as default security measures in modern browsers.

References

[1] Franco Callegati, Walter Cerroni, and Marco Ramilli. 2009. Man-in-the-Middle Attack to the HTTPS Protocol. *IEEE Security Privacy* 7, 1 (2009), 78–81. doi:10.1109/MSP.2009.12

[2] Ankita R Chordiya, Subhrajit Majumder, and Ahmad Y Javaid. 2018. Man-in-the-Middle (MITM) Attack Based Hijacking of HTTP Traffic Using Open Source Tools. In *2018 IEEE International Conference on Electro/Information Technology (EIT)*. 0438–0443. doi:10.1109/EIT.2018.8500144

[3] Jeremy Clark and Paul C. van Oorschot. 2013. SoK: SSL and HTTPS: Revisiting Past Challenges and Evaluating Certificate Trust Model Enhancements. In *2013 IEEE Symposium on Security and Privacy*. 511–525. doi:10.1109/SP.2013.41

[4] Aldo Cortesi, Maximilian Hils, Thomas Kriechbaumer, and contributors. 2010–. mitmproxy: A free and open source interactive HTTPS proxy. https://mitmproxy.org/ [Version 12.0].

[5] Xavier de Carné de Carnavalet and Mohammad Mannan. 2016. Killed by Proxy: Analyzing Client-end TLS Interception Software. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society. doi:10.14722/ndss.2016.23374

[6] Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. 2018. OPAQUE: An Asymmetric PAKE Protocol Secure Against Pre-computation Attacks. In *Advances in Cryptology – EUROCRYPT 2018*, Jesper Buus Nielsen and Vincent Rijmen (Eds.). Springer International Publishing, Cham, 456–486.

[7] Yogesh Joshi, Debabrata Das, and Subir Saha. 2009. Mitigating man in the middle attack over secure sockets layer. In *2009 IEEE International Conference on Internet Multimedia Services Architecture and Applications (IMSAA)*. 1–5. doi:10.1109/IMSAA.2009.5439461

[8] Ani Petrosyan. 2025. Number of data compromises and impacted individuals in U.S. 2005-2024. https://www.statista.com/statistics/273550/data-breaches-recorded-in-the-united-states-by-number-of-breaches-and-records-exposed/. [Online; accessed 15 Nov 2025].

[9] Douglas Stebila and Nick Sullivan. 2015. An Analysis of TLS Handshake Proxying. In *2015 IEEE Trustcom/BigDataSE/ISPA*, Vol. 1. 279–286. doi:10.1109/Trustcom.2015.385